

Claude Code + autoresearch

ML 엔지니어 없이 ML 모델 만들기

Jueun Seo · 2026. 04

Andrej Karpathy의 autoresearch 프레임워크를 빌려, Claude Code가 스스로 가설을 세우고 실험을 반복하는 방식으로 광고 매칭 ML 모델을 만든 경험을 공유합니다. 학습 데이터와 평가 지표 두 가지만 제대로 정의할 수 있다면, ML 엔지니어의 도움 없이도 간단한 ML 모델은 누구나 만들 수 있다는 이야기입니다.

배경: 경쟁사 리워드 광고 물량 분석

버즈빌에서는 6개 네트워크(buzzvil, nbt, tnk, adpopcorn, tapjoy, 당근마켓)의 리워드 광고를 크롤링하고 있습니다. 목적은 단순합니다 — 경쟁사에는 있고 버즈빌에는 없는 광고를 찾아내는 것.

이를 통해 미집행 광고 물량을 파악하고, 영업 기회를 발굴하고, 리워드 격차를 분석합니다.

Buzzvil Ad Intelligence

미집행 광고 분석 리포트 · 2026-04-15

버즈빌 미보유1140

리워드 격차136

액션 확장 기회104

리워드전체

~10P

11~500P

501~2000P

2001P~

네트워크전체

2개+

3개+

4개+

신규 2주

플랫폼전체

Android

iOS

149건

광고	중복	네트워크	리워드	CTA
A [달성하기] 뉴스캐시 회원가입	3	ADPOPCORN NBT TNK	ADPOPCORN 403P, NBT 450P, TNK 470P	최초 회원가입하면
A [달성하기] 블록 스마트: 퀘스트	3	ADPOPCORN NBT TNK	ADPOPCORN 448P, NBT 360P, TNK 330P	3,500점 달성하면
A [캐릭터생성] 파이널삼국지2 (최초오픈)	3	ADPOPCORN NBT TNK	ADPOPCORN 196P, NBT 270P, TNK 210P	캐릭터생성 완료하기
A [간편참여] 스톤에이지 키우기 - 모든 펫 증정!	3	ADPOPCORN NBT TNK	ADPOPCORN 105P, NBT 135P, TNK 110P	앱 내려받으면
A [간편참여] 황제의 검을 떨궂다	3	ADPOPCORN NBT TNK	ADPOPCORN 105P, NBT 135P, TNK 110P	앱 내려받으면
A [방문하기] 네이버플러스 스토어	3	ADPOPCORN NBT TNK	ADPOPCORN 1P, NBT 17P, NBT 1P, TNK 4P	상품을 확인하고 구매해보세요
A [달성하기] 머지 블록 랩	2	ADPOPCORN NBT	ADPOPCORN 448P, NBT 450P	3,500점 달성하면
A [회원가입] 머니러닝 회원가입	2	ADPOPCORN TNK	ADPOPCORN 336P, TNK 420P	최초 회원가입하면
A [달성하기] 주월드	2	ADPOPCORN NBT	ADPOPCORN 336P, NBT 315P	5,000점 달성하면
A NOL	2	NBT TNK	NBT 225P, NBT 105P, TNK 310P	앱 최초 접속하면
A 현대해상다이렉트 자동차보험	2	NBT TNK	NBT 200P, TNK 160P	내 차 보험 신규가입하면
A 맥로 메이커 · 과일 파티	2	NBT TNK	NBT 135P, TNK 180P	앱 최초 접속하면

Buzzvil Ad Intelligence 리포트. 경쟁사에만 있는 광고, 리워드 격차, 액션 확장 기회를 분석.

이 분석을 제대로 하려면 핵심적인 문제를 풀어야 합니다. 서로 다른 네트워크에 실린 **같은 광고를 어떻게 판별할 것인가?**

네트워크마다 광고 제목, CTA, 카테고리 포맷이 다릅니다. 예를 들어 같은 "Screw Out" 게임 광고라도:

Buzzvil

[글로벌 멀티액션] Screw Out
CTA: 참여하기 / 10,468P

TNK

Screw Out 3D: Nut Sort Jam(멀티액션)
CTA: 30일 이내 달성하셔야... / 3,840P

단순 문자열 비교로는 불가능합니다.

첫 시도: Claude Code에게 직접 분석 시키기

처음에는 Claude Code에게 광고 목록을 주고 직접 "같은 광고인지" 판단하게 했습니다. 충분히 똑똑하니까 잘하지 않을까 싶었는데, 실전에서는 문제가 있었습니다.

	LLM 직접 호출	경량 ML 모델
속도	수분 (API 호출)	수밀리초
비용	매 실행마다 토큰 소모	0원 (순수 JS)
일관성	같은 쌍도 결과가 달라질 수 있음	동일 입력 = 동일 출력
반복 사용	매번 비용 발생	무한 재사용

데이터가 많아서 시간이 오래 걸리고, 토큰을 많이 먹고, 결과물의 퀄리티도 일정하지 않았습니다. 그래서 결정했습니다 — **동일 광고를 판별하는 경량 ML 모델을 직접 만들자.**

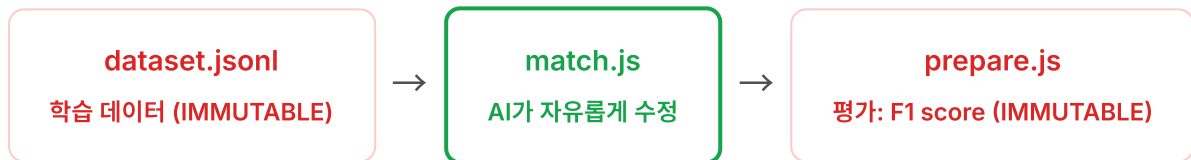
autoresearch 프레임워크 차용

Andrej Karpathy가 만든 **autoresearch**라는 프레임워크가 있습니다. LLM이 스스로 가설을 세우고, 코드를 수정하고, 실험을 돌리고, 결과를 평가하는 자율 연구 루프입니다. 원래는 GPU 위에서 language model pretraining의 val_bpb(validation bits per byte)를 최소화하는 용도인데, 이 구조를 광고 매칭 문제에 맞게 변형했습니다.

Claude Code에게 autoresearch 레포 위치를 알려주고 "이 구조를 보고 베껴서 우리 프로젝트에 맞는 `program.md` 를 만들어라"고 지시했습니다.

핵심 원리: 3개 파일

autoresearch의 핵심은 놀라울 정도로 단순합니다. 사람이 고정하는 것과 AI가 자유롭게 바꿀 수 있는 것을 명확히 분리하는 것입니다.



- **IMMUTABLE** — 학습 데이터(`dataset.jsonl`)와 평가 방법(`prepare.js`)은 사람이 정의하고 고정합니다.
- **EDITABLE** — 피처, 모델 구조, 하이퍼파라미터(`match.js`)는 AI가 자유롭게 변경합니다.

평가 결과는 **단일 스코어(F1)**로 나옵니다. 이 숫자가 올라갔는지 내려갔는지만 보면 되므로 AI가 자율적으로 "이 시도는 keep, 저 시도는 revert" 판단을 내릴 수 있습니다.

"NEVER STOP: 사람이 수동으로 중단할 때까지 계속 실험. 아이디어가 바닥나면 더 깊이 생각할 것."

— `program.md` (agent 지시서)

학습 데이터 만들기: LLM으로 라벨링

ML 모델을 만들려면 학습 데이터가 필요합니다. 옛날에는 라벨러를 고용해서 해야 했던 일인데, 이제는 LLM 성능이 충분히 좋아서 **Claude Code**에게 라벨링을 시킬 수 있습니다.

01 DB에서 후보 쌍 자동 생성 (브랜드 매칭, 동일 네트워크 등 전략별)

02 Claude Opus가 각 쌍에 대해 same_subject / same_action 판단 + 이유 작성

03 Codex로 교차 검증 — 불일치 케이스만 사람이 리뷰

04 Train / Eval split — Eval 결과는 에러 디테일을 숨겨 모델이 eval set에 과적합하지 않도록 방지

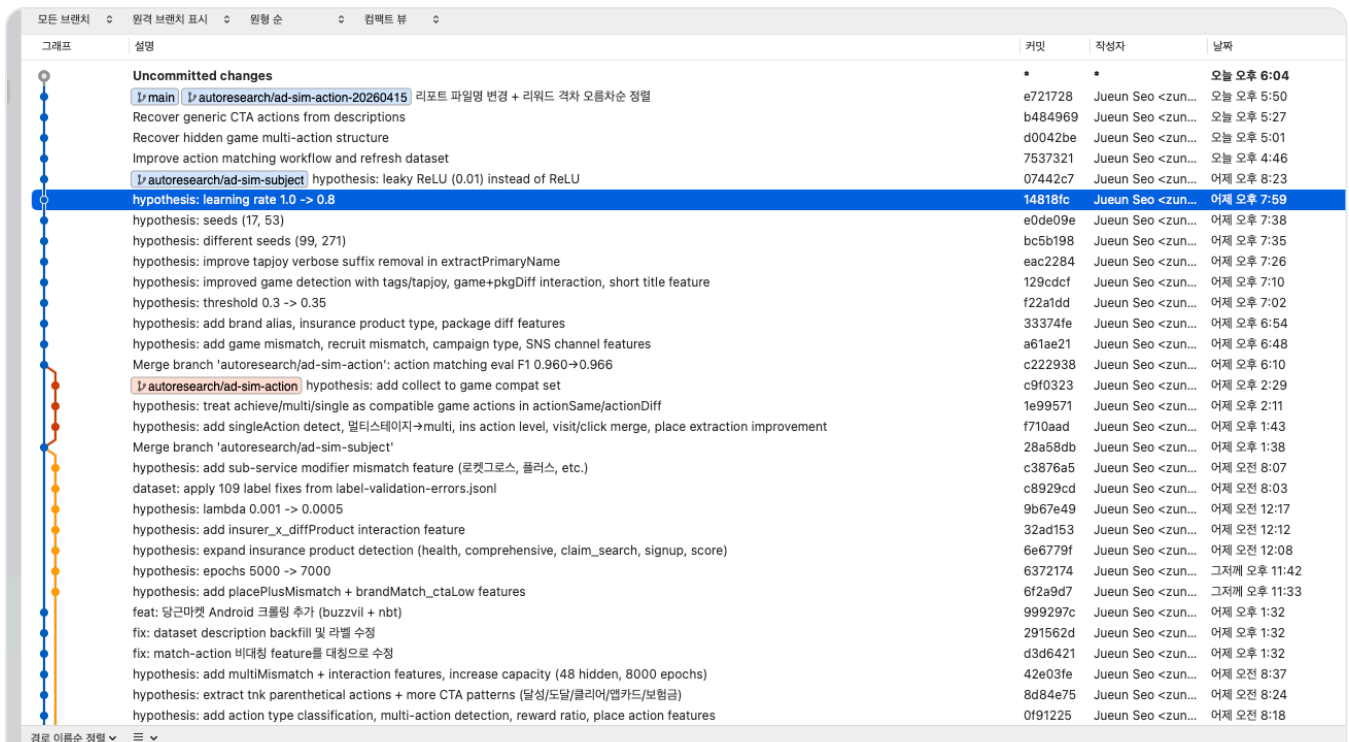
일종의 **모델 증류(distillation)**입니다. Opus급 대형 모델이 가진 "동일 광고 판별" 능력만 추출해서, 순수 JavaScript로 동작하는 작은 2-layer MLP로 압축하는 것입니다. 초기 1,316쌍으로 시작해서 현재 5,689쌍까지 확장했습니다.

자율 실험 루프

학습 데이터와 평가 프레임워크가 준비되면, Claude Code는 다음과 같은 루프를 무한 반복합니다:

1. **에러 분석** — Train FP/FN 패턴을 파악합니다. 왜 틀렸는지 원인을 분석합니다.
2. **가설 수립** — "보험사 이름은 같지만 상품이 다른 경우를 구분해야 한다" 같은 가설을 세웁니다.
3. **코드 수정 + 커밋** — `match.js` 를 수정하고 `hypothesis: ...` 메시지로 커밋합니다.
4. **평가** — `prepare.js` 를 실행해서 eval F1을 확인합니다.
5. **판단** — F1이 개선되면 **keep**(커밋 유지), 하락하면 **revert**(되돌림).

사람이 잠든 사이에도 실험이 계속 진행됩니다. 아래는 실제 git history입니다.



실제 git log. hypothesis: 로 시작하는 커밋들이 Claude Code가 자율적으로 실험한 흔적. 노란색 하이라이트는 autoresearch 브랜치가 merge된 시점.

결과

총 36개의 hypothesis 커밋이 기록되었습니다. 이 중 11개가 keep(성능 향상), 20개 이상이 revert(실패/퇴보)되었습니다. 성공률은 약 35%. 실패한 시도가 훨씬 많지만, keep된 것만 누적되므로 모델은 계속 좋아집니다.

Baseline 0.542

Subject 모델 0.960

Action 모델 0.955

최종 모델은 2단계 파이프라인입니다. "같은 광고주인가?"를 먼저 판단하고(matchSubject), "같은 행동을 요구하는가?"를 이어서 판단합니다(matchAction).

```
// 동일 광고 판별 = 2단계 파이프라인
function match(adA, adB) {
```

```

return matchSubject(adA, adB) === 1 // 같은 광고주/앱/서비스?
    && matchAction(adA, adB) === 1 // 같은 행동을 요구?
    ? 1 : 0;
}

```

46-47

hand-crafted 피쳐

0

외부 의존성

~2초

학습 시간 (모듈 로드 시)

솔직한 평가

잘된 점

- Opus 직접 호출 대비 **속도 1000배+, 비용 0원, 일관성 보장**
- 비-ML 엔지니어가 프레임워크 차용 + LLM 지시만으로 **실용적인 모델을 구축**
- 매일 자동 크롤링 후 즉시 분석이 가능해짐

한계

- 코드를 들여다보면 **도메인 특화 휴리스틱이 상당히 많음** — 보험사 추출, 게임 감지, 네이버 플레이스 액션 분류, 브랜드 alias 매핑 등. Claude Code가 feature engineering 을 하면서 도메인 룰을 직접 코딩한 것
- 순수 "학습"보다는 **"규칙 + 학습" 하이브리드**에 가까움. 개선 여지가 있는 상태

당신도 할 수 있다

이 경험에서 핵심은 두 가지입니다.

1

학습 데이터. Claude Code를 쓰면 많은 경우 쉽게 만들 수 있습니다. LLM 라벨링 + 별도 모델로 교차 검증.

2

평가 지표. 모델의 성능을 정확히 측정하는 단일 스코어를 정의할 수 있으면 됩니다. "이 숫자가 올라가면 좋은 것."

이 두 가지만 준비하면, 나머지(모델 구조, 피쳐, 하이퍼파라미터)는 Claude Code가 자율적으로 탐색합니다.

그동안 ROI가 안 나와서, 시간이 없어서 못했던 ML 프로젝트가 있다면 시도해볼 만합니다. 예를 들면:

어뷰저/정상유저 식별

CS 자동 처리를 위한 유저 분류. 과거 CS 데이터로 학습 데이터 구축 가능.

간단한 추천 로직

유저 행동 기반 콘텐츠/광고 추천. 클릭률 데이터가 이미 있으므로 평가 지표 명확.

이탈 예측

유저가 이탈할 시점을 예측. 과거 이탈 데이터에서 패턴 학습.

광고 성과 예측

새 광고의 예상 CTR/전환율. 기존 성과 데이터로 학습.

학습 데이터 + 평가 지표 = 누구나 ML 모델

Claude Code · autoresearch framework · Jueun Seo · 2026. 04